

通过高效元内核上的混合预填充/解码/验证调度，通过 SOTA 优化解决生产 LLM 服务系统中的动态性

Mingcong Song*
Huawei

Xinru Tang*
Tsinghua University

Fengfan Hou
Huawei

Jing Li
Huawei

Wei Wei
Huawei

Yipeng Ma
Huawei

Runqiu Xiao
Huawei

Hongjie Si
Huawei

Dingcheng Jiang
Tsinghua University

Shouyi Yin
Tsinghua University/Shanghai AI Lab

Yang Hu
Tsinghua University

Guoping Long
Huawei

抽象的

满足生产级大型语言模型 (LLM) 服务系统对低延迟和成本效率日益增长的需求需要集成先进的优化技术。然而，LLM 的动态且不可预测的输入输出长度，再加上这些优化，加剧了工作负载可变性的问题，使得人工智能加速器难以保持高效率，尤其是具有基于图块的编程模型的 DSA。为了应对这一挑战，我们推出了 XY-Serve，这是一种多功能、Ascend 原生、端到端生产 LLM 服务系统。核心思想是一种抽象机制，通过将计算分解为统一的、硬件友好的、细粒度的元原语来平滑工作负载的可变性。为了引起注意，我们提出了一个元内核，它使用架构感知的图块大小来计算 matmul-softmax-matmul 的基本模式。对于 GEMM，我们引入了一种虚拟填充方案，该方案适应动态形状变化，同时使用具有各种固定图块大小的高效 GEMM 基元。XY-Serve 与 vLLM 和谐相处。实验结果显示，与目前公开的升腾 NPU 基准相比，端到端吞吐量提高了高达 89%。此外，相对于现有库，我们的方法优于现有的 GEMM（平均快 14.6%）和注意力（平均快 21.5%）内核。虽然这项工作是在 Ascend 原生的，但我们相信该方法也可以轻松应用于 SIMT 架构。

1 简介

大型语言模型 (LLM) [41, 42] 已经取得了令人印象深刻的准确性，并广泛应用于自然语言处理[19]和计算机视觉[21, 25]等领域。如图1(a)所示，LLM推理通常由两个阶段组成：预填充和解码。在预填充阶段，LLM 处理用户的输入以生成初始令牌，同时缓存键/值 (K/V) 数据以供将来使用。

*These authors contributed equally to this work.

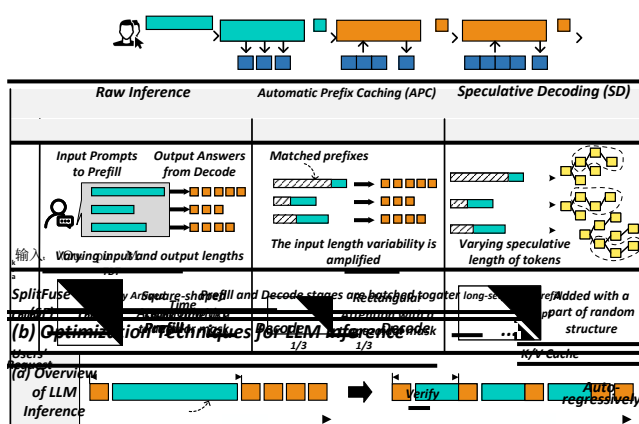


图 1: LLM 推理的动态。

在解码阶段，以自回归的方式顺序生成令牌。尽管法学硕士的性能令人印象深刻，但其计算成本和延迟却很高。随着模型大小的增加和输入序列变得更长，计算需求大幅增长，使得在线推理变得越来越具有挑战性[30]。

为了应对这些挑战，出现了许多优化技术来降低推理成本和延迟，例如自动前缀缓存 (APC) [8, 47]、推测解码 (SD) [20, 22, 32, 33, 38, 46] 和 SplitFuse [17, 18, 26]。APC 使具有匹配前缀的新查询能够重用缓存的 K/V 数据并跳过共享段的计算，从而提高预填充性能。SD 中采用草稿模型为目标模型每步生成多个令牌提供了机会，增强了键/值缓存和模型权重重用，这有助于缓解草稿模型中解码的内存限制瓶颈。SplitFuse 将长序列预填充令牌分割成较小的块，并将它们与解码令牌一起调度，从而减少对解码阶段的中断。

虽然这些优化有望提高推理效率，但它们也带来了新的复杂性。对于大语言模型

在生产系统服务中，由于缺乏友好的 SIMT 编程模型，关键挑战是如何将所有这些优化有效地集成到 AI 加速器上。我们在图 1(b) 中说明了这个问题。性能已经与不可预测的和变化的输入/输出令牌长度作斗争，这些优化可能会进一步加剧这个问题。

例如，APC 增加了输入提示长度的可变性，因为缓存的前缀数量取决于查询历史记录和实时内存可用性。使用 SD，解码阶段不再一次处理一个令牌。相反，它处理动态变化的推测长度的令牌[37]。这个新阶段称为验证阶段 [4, 45]。SD 还将注意力掩码从标准因果掩码转换为更复杂、动态生成的版本。此外，SplitFuse 将预填充和解码阶段合并到一个批次中，从而使一个批次内多个阶段的管理进一步复杂化。

这些动态性给 LLM 计算带来了重大挑战，特别是在线性和注意力模块中。首先，输入长度的不确定性导致线性运算中的任意矩阵形状，使旨在实现峰值计算效率的优化工作变得复杂。其次，APC、SD 和 SplitFuse 等技术的采用引入了注意力形状和掩模结构的更多样性，削弱了现有注意力内核的有效性。

此外，在实际系统中，预填充(P)、解码(D)和验证(V)阶段可以独立操作或组合操作。即使在分解部署中，D 节点也可以同时运行 D 和 V 阶段。如果分解部署[27,28,39,48]节点支持动态角色切换，则在角色转换期间也可能出现混合 P/D/V 组合问题。由于这些阶段在注意力阶段呈现出不同的计算负载，因此尝试枚举和优化每个可能的阶段组合成为一项劳动密集型且不切实际的任务。

为了应对动态性带来的挑战，我们推出了 XY-Serve，这是一个多功能、Ascend 原生、端到端生产的 LLM 服务系统。主要思想是引入一个抽象来弥合不同的高级工作负载和固定的硬件友好的低级元基元之间的差距。具体来说，XY-Serve 具有令牌明智的调度机制，可以将令牌分批处理。令牌可以来自预填充、验证和解码阶段。然后，令牌块将由三个核心组件处理：工作负载分解、计算任务重新排序和元内核。

工作负载分解是一种将动态工作负载分解并映射到硬件友好的元原语的机制。对于注意力计算，它通过动态平铺统一 P/D/V 阶段，生成硬件友好的基于平铺的计算任务。每个任务都会计算一个基本的 matmul-softmax-matmul 模式。对于 GEMM，它将动态形状的 matmul 运算分解为一组具有固定图块大小的基本 matmul 原语。这种转变似乎

看似简单，但在没有任何实际填充开销的情况下很难实现。我们引入了一种新颖的虚拟填充机制来解决这个问题，而不会在基于图块的编程模型的人工智能加速器上引入任何额外的开销。

分解后，计算任务重新排序重新组织分解的任务并将其调度给硬件处理核心。重新排序的目标是在细粒度级别平滑不同的任务大小，平衡不同核心上的工作负载以获得高执行效率。对于注意力来说，P/D/V 任务具有截然不同的粒度；因此，对分解的任务进行重新排序对于实现负载平衡至关重要。GEMM 的情况有所不同；任务重新排序是改善 L2 缓存局部性和缓解存储体冲突的有效方法。

通过工作负载分解和任务重新排序，可以构建有效的注意力和 GEMM 内核。为了引起注意，我们提出元内核来有效地并行化具有不同图块大小的 matmul-softmax-matmul 模式的计算，而不需要区分它们源自哪个 P/D/V 阶段。通过这种解耦方法，可以无缝、高效地集成一系列优化，包括 APC、PageAttention [31]、SplitFuse、SD 和 FlashAttention [24]。这些优化在我们的元内核中协同工作，实现卓越的性能。

此外，我们的注意力元内核配备了积极的片上 Cube Core-Vector Core 编排管道，以及动态处理各种注意力掩码的新颖方案。这些低级技术最大限度地减少了片外存储器访问，并最大限度地提高了片上工作负载平衡和执行效率。对于 GEMM 操作，我们首先设计并实现一组具有固定图块大小的高效元内核。为了动态处理任意输入形状，我们在片上内存级别采用虚拟填充，并结合选择性 HBM 读取和写入，并且不引入任何实际填充开销。换句话说，我们的方法允许矩阵计算无缝处理动态形状，同时保留固定形状优化的性能优势。

实验评估表明，我们的注意力内核在生产工作负载中实现了更高的效率，比现有实现 (torch-npu PFA [12] 和 IFA [11]) 平均高出 21.5%。我们的 GEMM 内核不仅支持任意矩阵形状，而且与现有基线相比，性能平均提高了 14.6%。在端到端评估中，XY-Serve 在公开数据集 [13] 上比 Ascend-vLLM [14] 提高了 89%。虽然目前在升腾 NPU 上实施 [34, 35]，但这些技术也可以应用于其他人工智能平台。最后，我们对基于 GPU 的推理系统进行了比较评估。就端到端 MFU 和 MBU 而言，XY-Serve 的性能可与最佳 GPU 实现相媲美。

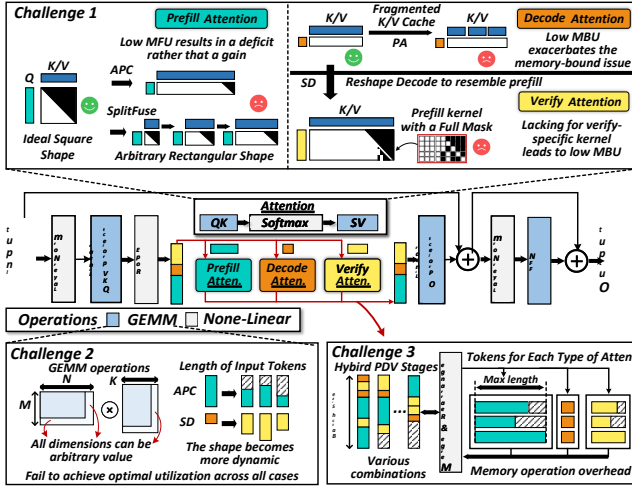


图 2：动态工作负载带来的挑战。

2 动机

在本节中，我们探讨了线和注意力模块中动态工作负载带来的挑战，并分析了处理 P/D/V 混合阶段的复杂性，如图 2 所示。最后，我们讨论了采用基于分块的编程模型的人工智能加速器（例如华为 Ascend NPU [34, 35]）在支持动态工作负载方面面临的其他挑战。

2.1 多元化关注

APC、SD和SplitFuse等新技术的引入增加了Attention形状和掩模结构的多样性，导致当前NPU注意力内核（torch-npu 2.1 FusedInferAttentionScore [10]）的MFU和MBU问题，如图3所示。

首先，对于预填充注意力，在没有任何优化的情况下，查询长度等于键值长度，从而产生方形注意力得分矩阵和下三角掩模。这种形状非常适合优化[36]，因为可以利用掩模中的稀疏性来实现高性能。目前最先进的 (SOTA) torch-NPU 内核可以有效地处理此类带有三角形掩模的方形输入，实现 53% 的 MFU。

然而，在涉及前缀的现实场景中，部分键和值会从 K/V 缓存中重用 [8, 47]，将注意力分数形状从正方形转换为任意尺寸的矩形。对于这些矩形形状，torch-npu 内核的性能显著下降，MFU 分别下降至 47% 和 30%。虽然 K/V 缓存的重用理论上可以减少计算量，但内核效率的下降抵消了这一好处，导致没有有意义的端到端性能提升。

其次，与 Linear 不同，Linear 中的代币来自不同的地方。

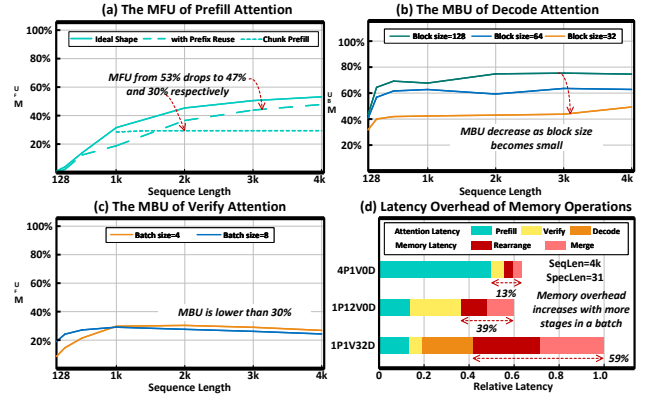


图 3：Attention Kernel 的 MFU 和 MBU。

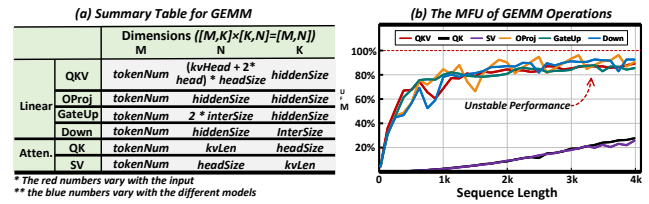


图 4：大语言模型 MFU 中的 GEMM 操作。

批次可以共享权重，解码Attention需要每个token有自己独立的K/V缓存，使得重用变得不可能。这导致计算本质上受内存限制。PagedAttention (PA) [31] 进一步分割 K/V 缓存访问模式，限制对小块大小部分的访问，而不是对大的连续块的访问。如图3 (b) 所示，MBU随着块大小变小而减少，加剧了解码阶段的内存瓶颈。

最后，推测解码 (SD) [20,33,38]将解码注意力重塑为类似于预填充注意力，但查询长度更短。不同的 SD 算法生成具有不同因果结构的令牌，从而在验证阶段产生不同的掩码。然而，由于缺乏针对 Ascend 的验证特定内核，因此使用配备完整掩码的预填充注意力内核作为后备。此回退会阻止特定于验证的优化，导致 MBU 低于 30%。

2.2 动态GEMM

GEMM 是大语言模型核心操作。除了线性部分中的四个 GEMM 操作之外，Attention 中的查询键 (QK) 和得分值 (SV) 计算也依赖于 GEMM。我们在图 4 (a) 中总结了所有 GEMM 操作。对于线性运算，M 维度与标记的数量相关。对于注意力 QK 和 SV 操作，N 和 K 维度取决于 token K/V 长度。如前所述，预填充和验证阶段的令牌数量是高度动态的，并且这种可变性进一步

通过引入自动前缀缓存 (APC) [8, 47] 放大了这一点, 使令牌长度更加不可预测。

此外, 对于线性 GEMM, 维度 N 和 K 受 *hiddenSize* 的影响, 而 *hiddenSize* 随 LLM 的架构而变化。因此, LLM GEMM 操作中的 M 、 N 和 K 维度都是动态变化的。

在人工智能加速器上设计矩阵乘法以支持任意形状, 同时确保高性能本身就具有挑战性。大多数加速器通常针对特定的图块大小进行优化, 例如 16×16 , 这使得很难充分利用其对任意形状的功能。不规则的形状使并行性和负载平衡变得复杂, 导致跨处理单元的工作负载分配效率低下。对灵活算法适应不同形状的要求增加了算法的复杂性。此外, 处理非均匀矩阵大小的边界条件会带来开销, 进一步影响性能。如图4 (b) 所示, 采用通用GEMM内核 (torch-npu 2.1线性算子[9]) 来适应不稳定性能的所有可能结果, 并且无法在所有GEMM操作中实现最佳利用率。

2.3 混合 P/D/V 阶段

在实际系统中, P/D/V阶段可以独立存在或同时共存, 从而导致在给定的调度预算内交错的P/D/V阶段的任意组合。对于线性运算, 来自不同阶段的令牌可以组合在一起, 并被视为 GEMM 运算中的左矩阵, 在右侧共享相同的权重矩阵。这种方法很简单, 因为来自不同阶段的令牌可以重用相同的大模型权重。

相比之下, 处理注意力操作要复杂得多。阶段是独立的, 即使在同一个阶段内, 批次也是独立的。对每个可能的阶段和批次组合进行枚举和定制优化是高度劳动密集型且不切实际的。

一种常见的替代方案是逐批执行, 例如选择性批处理[44], 它通过调用相应的注意力内核来独立处理每个批次。然而, 这种方法会因分割、重新排列和合并数据而引入额外的内存开销, 从而降低整体系统性能。如图3(d) 所示, 内存开销可能占50%以上。此外, 逐批处理导致计算资源利用效率低下, 进一步限制了系统效率。

2.4 升腾NPU微架构

华为Ascend NPU基于华为达芬奇架构[34, 35]构建, 是一款高性能AI处理器。图 5 展示了 Ascend 的微架构, 主要由 AIC 和 AIV 组件组成。AIC 类似于 Nvidia 的 Tensor Core, 处理矩阵计算, 而

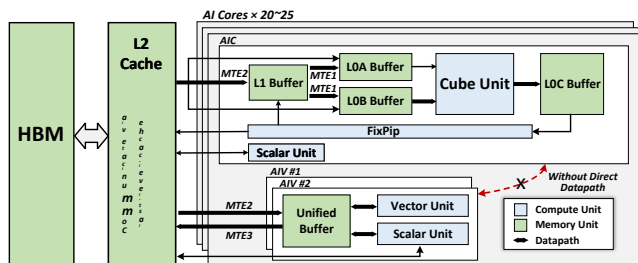


图 5: Ascend 910B 的微架构。

AIV 负责矢量操作。AIC 和 AIV 是分开的, 没有直接的数据路径, 因此在设计混合内核时确保它们的数据交互通过 L2 缓存进行至关重要。与 GPU 相比, NPU 具有更大的核心粒度, 使得核心之间的负载平衡更加关键。内存传输引擎 (MTE) 处理数据移动。无论是 AIC、AIV 还是 MTE, 所有数据管理和传输都发生在图块级别。具有基于图块的编程模型的人工智能加速器越来越受到社区的关注。然而, 这种基于图块的处理模型在管理以令牌粒度变化的动态工作负载时遇到了重大挑战。典型的解决方案涉及复杂的填充/取消填充操作, 这会导致计算和内存的浪费, 从而导致大量的开销。

3 XY-Serve概述

为了解决上述挑战, 我们开发了 XY-Serve, 这是一个多功能的端到端生产 LLM 服务系统, 它基于四个关键组件构建: Token-wise Scheduling、Dynamic Task Decomposition and Reordering、MetaAttention 和 SmoothGEMM。

3.1 Token-wise调度

当用户的请求进入系统时, 它首先通过 APC 模块, 该模块将传入的提示与 K/V 缓存中的现有提示进行匹配, 从而实现令牌明智的重用。任何不匹配的令牌都会添加到调度队列中。因此, 调度队列中的提示长度就是用户的输入长度减去 K/V 缓存中已经缓存的令牌长度。由于用户的输入和缓存的令牌长度都是动态可变的, 因此调度队列中的提示长度变得更加动态。调度队列还包括来自先前等待处理的请求的解码和推测令牌。

我们的调度系统从调度队列中选择固定预算长度的令牌来形成块, 其中可能包括来自预填充、验证或解码阶段的令牌。为了改善第一个令牌延迟, 预填充请求被优先考虑。如果用户的预填充提示超出预算长度, 则会分为

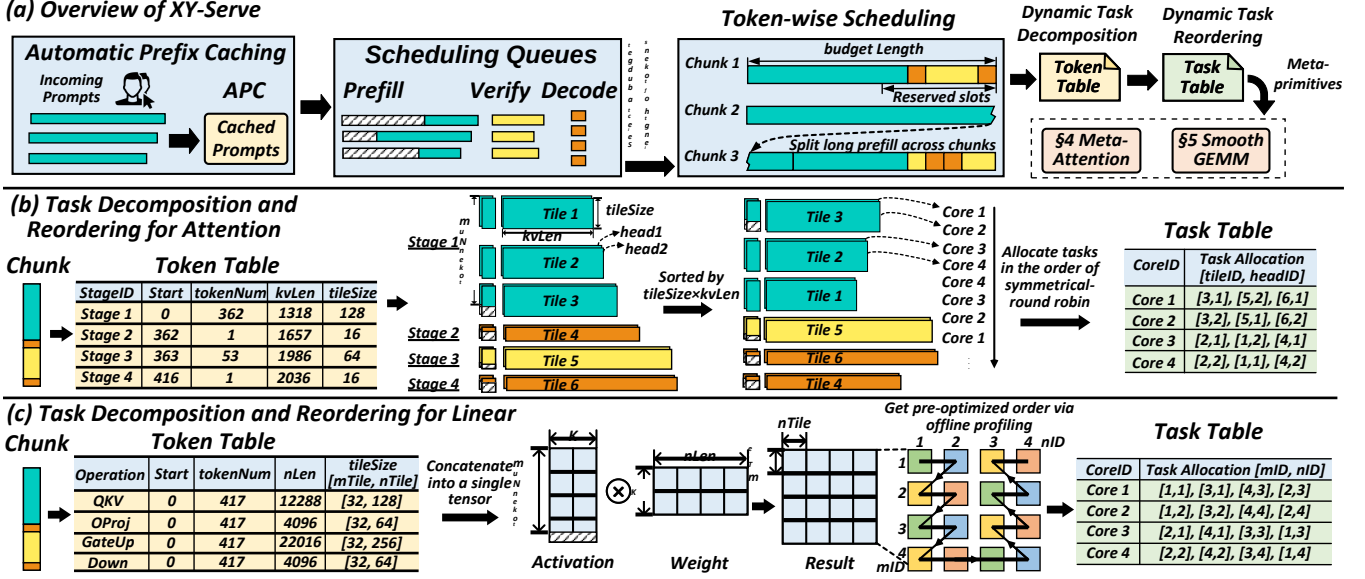


图 6: XY-Serve 概述。

较小的部分，以确保每个计划的块保持在预算之内。为了最大限度地减少解码时预填充造成的中断并保持稳定的令牌之间时间 (TBT)，为解码和推测令牌保留了某些时隙。仅当队列没有解码或推测令牌时才调度仅预填充块。

如图 6 (a) 所示，单个块内的 P/D/V 阶段的组成本质上是不可预测的，每个阶段的令牌计数任意变化，并且每个阶段具有不同的历史 K/V 长度。此外，块中的总令牌计数可能并不总是与预算长度匹配，并且在低系统负载下可能会低于预算。为了解决四个级别的动态问题，我们的调度完全按照单个代币的粒度进行操作，而不关心其来源。

3.2 任务分解

虽然令牌明智的调度通过减少泡沫和优化资源利用率来提高效率，但它引入的四个级别的活力对执行提出了重大挑战，特别是在具有基于图块的编程模型的人工智能加速器上。

为了应对这些挑战，我们提出了一种动态分解机制，将动态工作负载转换为硬件友好的、基于图块的计算单元。使用令牌表，每个阶段都在逻辑上分解为瓦片块。在图块级别，计算模块可以并行处理这些块，而无需区分它们的 P/D/V 阶段来源。重要的是，这种平铺分解纯粹是逻辑性的，不需要更改物理数据布局。

在接下来的章节中，我们将详细分析

如何将平铺分解应用于注意力层和线性层。

3.2.1 注意力分解

注意，令牌表包含每个 P/D/V 阶段的条目，每个条目指定关键属性，包括 *stageID*、*start position*、*tokenNum* 数量、历史 *kvLen* 和 *tileSize*。如图 6 (b) 所示，stage-1 (P) 被分解为 3 个平铺块，而 stage-2 和 stage-4 (D) 各被分为 1 个平铺块，stage-3 (V) 也被分解为 1 个平铺块。每个切片块由 *headNum* 个切片单元组成，从而在切片级别总共有 $6 \times headNum$ 个切片单元。

3.2.2 线性分解

对于线性运算，由于来自不同阶段的令牌可以共享相同的权重，因此它们被连接成一个大张量并乘以共享权重。这种方法避免了多次 GEMM 调用，增强了权重重用，并简化了计算。因此，线性运算不需要在阶段之间进行区分，并且可以统一处理。

在令牌表中，每个线性运算符对应于 6(c) 中所示的单个条目。四个主要线性运算是 *QKV*、*OProj*、*GateUp* 和 *Down*。对于这些操作，*start position* 设置为 0，表示所有标记都从级联张量的开头开始处理。*tokenNum* 等于当前调度块中的令牌总数。对维度为 $tokenNum \times nLen$ 的结果矩阵执行平铺，其中每个平铺块对应于结果的一个子矩阵。每个瓷砖块

被分配给单个人工智能核心，允许不同的核心并行处理不同的块。每个线性运算符都有自己特定的 *tileSize*，由运算的 *tokenNum* 和 *nLen* 形状决定。

3.3 任务重新排序

将动态工作负载从 P/D/V 混合阶段分解为基本瓦片单元后，有必要对这些瓦片单元重新排序并生成任务表以增强性能。任务表负责将这些瓦片单元调度到硬件上，每个条目指定一个 *coreID* 以及分配给该核心的瓦片列表。基于这个任务表，注意力和线性可以根据它们的 *coreID* 简单地检索相应的图块单元。这种方法不仅最大限度地提高了硬件效率，还简化了注意力和线性内核的设计。

3.3.1 注意力重新排序

在各个阶段执行动态平铺后，生成的平铺显示不同的 *tileSize* 和 *kvLen* 值。这种变化可能会导致并行处理期间的负载不平衡。为了解决这个问题，我们将每个图块的计算负载计算为其面积，定义为 $tileSize \times kvLen$ 。

为了高效调度，图块最初根据其计算负载从最大到最小进行排序。随后，这些块以对称循环方式分配给 AI 核心。如图6 (b) 所示，假设有四个AI核心，分片单元按照core-1、core-2、core-3、core-4、core-4、core-3、core-2、core-1等顺序分配，重复这种模式以确保计算任务的平衡和高效分配。

该任务调度信息存储在任务表中并传递给注意力模块。任务表引导注意力模块利用头部和图块尺寸有效地执行并行处理。这种机制可确保跨 AI 核心的平衡计算，从而最大限度地提高硬件利用率。

3.3.2 线性重排序

鉴于仅支持有限的固定形状集，我们执行离线优化以确定线性操作的最有效的任务分配策略。这涉及对每种形状进行分析和自定义任务分配，以最大限度地提高性能。优化的策略被存储以供在执行期间使用。在运行时，XY-Serve 利用令牌表来识别当前形状，并使用此信息来检索相应的预优化任务表。然后，任务表被传递到线性模块，指导其以优化的方式执行任务。

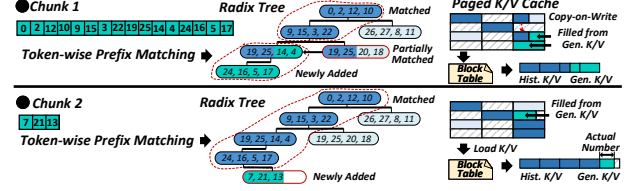


图 7: 按令牌方式的 K/V 缓存重用。

4 元注意力

在本节中，我们首先解释我们的注意力模块如何支持 A PC、分块预填充和 SD 等高级功能。然后，我们描述如何优化注意力性能以将其推向硬件极限。

4.1 元注意力设计

4.1.1 处理 Token-wise 处理

支持前缀重用和分块预填充的核心要求是注意力模块必须能够处理任意 K/V 缓存长度并执行 token-wise K/V 缓存重用。为了实现这一点，我们使用基数树来有效地管理 K/V 缓存。如图7所示，基数树中的每个节点代表一个K/V缓存块。基数树允许快速匹配历史K/V缓存块。当特定节点发生不匹配时（即其对应的 K/V 缓存块仅包含部分匹配），我们使用写时复制机制创建一个新块，将新数据刷新到该块中，然后将该块添加回基数树中。如果在这种不匹配之后生成了额外的新块，则这些块将直接作为不匹配块的子节点插入到基数树中。

该机制有效地管理历史和新生成的 K/V 数据，并使用写时复制将它们无缝合并，以确保 K/V 缓存的连续性。在预填充注意力过程中，根据块表读取相应的 K/V 块（包括历史块和新生成的块）。我们还跟踪最后一个区块中代币的实际数量，确保准确的代币处理。

此外，我们的系统可以自动将历史 K/V 数据缓存为前缀，从而使用户无需手动指定它们。系统允许用户设置历史K/V数据缓存量的上限。一旦达到此限制，或者当新的 K/V 数据需要空间时，系统将自动从基数树的叶节点中驱逐最近最少使用的 K/V 块。

4.1.2 最小化推测解码的掩码

推测执行可以分为两种类型：基于序列的推测[20,33,38,46]和基于树的推测[22,32]。基于序列的推测产生

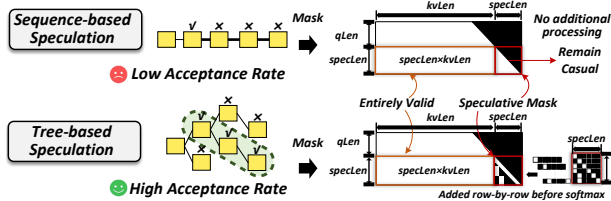


图 8: 推测解码算法。

单个序列中的多个标记；然而，其接受率普遍较低。相比之下，基于树的推测算法同时生成多个序列的预测，并将它们组织在树结构中。这种做法可以进一步提高投机的接受率。

两种类型都需要支持任意推测长度。此外，基于树的推测使用更复杂和动态的掩码，这不太适合基于矢量的硬件。通过分析该掩模的结构，我们可以识别能够实现高效处理的规律。

如图 8 所示，推测解码通过引入 $specLen \times (kvLen + specLen)$ 区域扩展了标准预填充 ($qLen \times kvLen$) 的因果掩码。在此区域内， $specLen \times kvLen$ 完全有效，而只有 $specLen \times specLen$ 部分需要特殊处理，称为“推测掩码”。

对于基于序列的推测，推测掩模仍然是因果的，不需要额外的处理，允许直接应用我们的无掩模方法。在基于树的推测中，我们仅在外部分生成掩码的 $specLen \times specLen$ 部分，然后将其传递到内核并应用于相应的注意力分数矩阵。

我们的设计逐行处理推测掩码，从而能够精确控制每行掩码的起始位置和长度。该方法有效地支持任意推测长度。调整掩模后，后续计算将遵循标准预填充过程。使用推测掩码作为中介，我们可以高效、无缝地支持各种推测算法。

4.2 元注意力优化

4.2.1 基于平铺的立方体向量编排

为了实现立方体和向量单元的并行执行，我们提出了一个管道，如图 9 所示。它确保中间数据传输仅通过 L2 缓存进行，避免昂贵的 HBM 访问。立方体单元负责 QK 和 SV 计算，而向量单元执行 $Softmax$ 。如果按顺序处理 ($QK \rightarrow Softmax \rightarrow SV$)，一次只有一个单元处于活动状态，从而导致效率低下。为了解决这个问题，我们采用了如图 10(a) 所示的流水线方法：完成第一个平铺数据的 QK 计算后，启动其 $Softmax$ 计算，同时

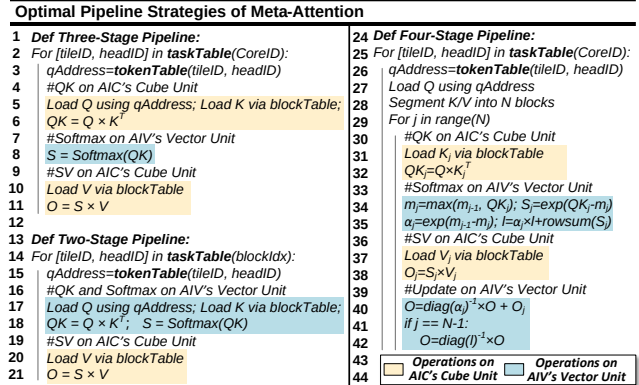


图 9: 立方体向量编排算法。

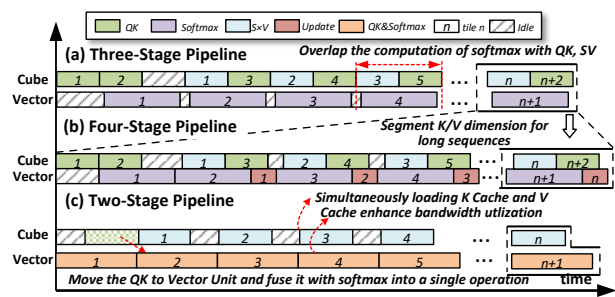


图 10: 元注意力管道。

同时开始第二个平铺数据的 QK 计算。这种重叠可确保立方体和矢量单元同时工作，从而最大限度地提高硬件利用率。

每个图块的中间数据大小是 $tileSize \times kvLen$ 。当同时处理 $coreNum$ 个图块时，所需的总中间缓冲区大小为 $pipeDepth \times tileSize \times kvLen \times coreNum$ 。通过确保该缓冲区适合 L2 缓存，我们可以避免访问 HBM。如果序列长度较短且中间结果适合 L2 缓存，则无需沿 K/V 维度进行分割。在这种情况下，可以采用三级流水线，避免序列分割所需的额外计算和更新。

然而，对于极长的序列，由于 L2 缓存的限制，沿着 K/V 维度的分割变得必要。这引入了一个额外的计算阶段，其中 $Softmax$ 操作分为两个步骤： $Softmax$ 和 $Update$ 。该管道扩展到四个阶段： $QK \rightarrow Softmax \rightarrow SV \rightarrow Update$ 。如图 10(b) 所示，我们最初预取一块数据的 QK 和 $Softmax$ 计算。随后，立方体调度在 QK 和 SV 任务之间交替，而向量调度在 $Softmax$ 和 $Update$ 任务之间交替。该设计支持立方体和矢量单元的并行操作，最大限度地提高硬件资源利用率，并确保短输入序列和长输入序列的最佳性能。

对于完全基于解码的任务，我们可以采用新的管道

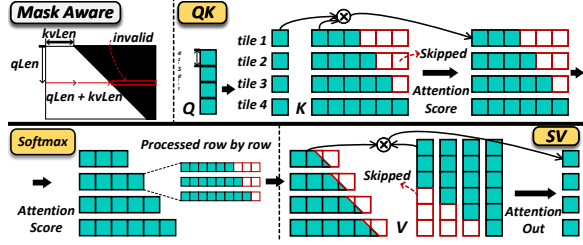


图 11: 掩模感知计算。

设计进一步优化性能并解决解码的内存限制问题。由于解码阶段的查询仅包含单个标记，因此 *Query* 矩阵被简化为向量。因此，*QK* 和 *SV* 计算从一般矩阵运算转变为矩阵向量运算。在立方体单元上执行这些操作会导致资源利用效率低下。为了解决这个问题，我们将 *QK* 运算移至向量单元，并将 *QK* 和 *Softmax* 运算融合为单个运算符。这种融合算子确保向量单元的执行时间与立方体单元执行 *SV* 操作所需的时间紧密结合，从而有效地平衡工作负载。

如图10 (c) 所示，管道图显示，当矢量单元对图块 *n* 执行 *QK* 和 *Softmax* 操作时，立方体单元同时对 *tile n-1* 执行 *SV* 操作。此外，立方体和向量单元可以同时访问 HBM 中存储的 *K/V* 缓存数据，提高内存带宽利用率，进一步提升整体性能。

4.2.2 利用掩模稀疏性

为了充分利用注意力掩模中的稀疏性并跳过冗余计算，我们采用了掩模感知策略，可以显著提高效率。如图11所示，在注意力计算中，一旦查询维度坐标索引 *qLen* 和 *K/V* 的 *kvLen* 已知，则注意力得分矩阵每行中 *qLen + kvLen* 之后的位置对应的任何数据都是无效的。这种洞察使我们能够指导 *QK*、*Softmax* 和 *SV* 操作来跳过这些无效区域中的计算。

在 *QK* 操作中，这个原理使我们能够直接省略相应结果的计算，有效地跳过结果矩阵中的红色平铺块。对于 *Softmax* 操作，由于它是逐行计算的，因此我们可以在 *token* 粒度上精确控制哪些注意力分数包含在计算中。这种令牌级控制使我们能够支持任意形状的掩码，同时保持无掩码设计，消除用户在外部分生成和传递掩码的开销。对于 *SV* 操作，跳过的计算发生在沿 *K* 维度求和期间，其中可以根据稀疏模式排除某些平铺块。通过应用这些

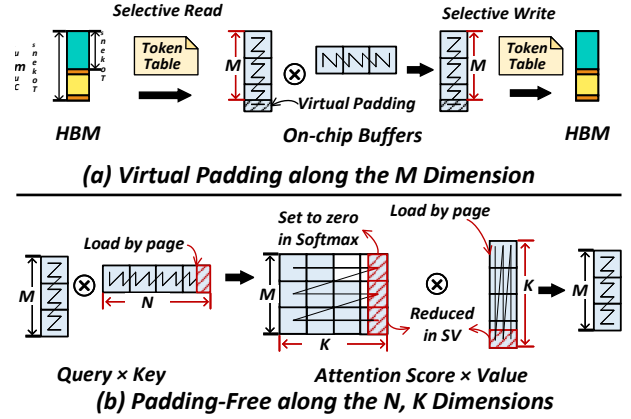


图 12: 处理任意形状的 GEMM 操作。

通过 *QK*、*Softmax* 和 *SV* 阶段的优化，我们有效地利用了掩模稀疏性，确保了高效且灵活的注意力机制。

5 平滑 GEMM

正如前面所讨论的，设计一个支持任意形状的矩阵乘法运算，同时在所有可能的形状上保持高性能是一个重大挑战。为了解决这个问题，我们采用了内存计算协同设计策略。我们不是针对每种可能的形状优化矩阵乘法，而是专注于最大化固定形状的性能。为了有效地处理任意形状，我们在片上内存级别引入了虚拟填充以及选择性读写机制。这种方法允许矩阵计算无缝地适应各种形状，同时仍然受益于固定形状优化的性能优势。

5.1 M 维度上的虚拟填充

如图12 (a) 所示，GEMM 中的维度 *M* 与令牌的数量本质上相关。在 Linear GEMM 的情况下，*M* 维度对应于 *P/D/V* 阶段的累积 *token* 计数，而在 Attention GEMM 中，它由每个阶段的 *token* 计数决定。在基于立方体或基于张量的人工智能加速器中，矩阵计算通常受到最小切片大小的限制（例如，立方体核心为 16×16 ）。常见的做法是填充 *M* 尺寸以与平铺尺寸的倍数对齐。然而，这种动态填充会带来不小的内存开销并降低性能。

为了缓解这些问题，我们将全局内存中的物理填充替换为芯片上的虚拟填充，并结合图 12 (a) 所示的选择性读写机制。这种方法可以有效地处理任意形状的矩阵。具体来说，片上缓冲区分配

以平铺大小的单位制成，以充分利用硬件的计算潜力。在数据从全局存储器传输到片上缓冲区期间，选择性读取操作仅复制实际的非填充数据。类似地，选择性写入操作确保只有非填充输出被写回全局存储器。

由于虚拟填充区域不会干扰非填充区域的计算结果，因此该方法保证了矩阵计算结果的正确性。此外，通过将计算限制为固定形状矩阵乘法，我们可以对这些固定形状应用高度定制化的优化，从而实现动态工作负载的高效率和灵活性。

5.2 N 和 K 维度的优化

如图4（a）所示，线性运算中 N 和 K 的维度由模型结构决定。这些尺寸通常是硬件图块大小的倍数，例如 16，从而无需额外的填充。

对于 Attention GEMM 操作，维度 N 和 K 与序列长度相关，可以采用任意值。理论上，这些尺寸需要填充。然而，这自然是由 K/V 缓存的块页结构处理的，该结构在与图块大小的倍数对齐的块中存储和读取数据。如图 12（b）所示，通过读取整个块，读取长度本质上符合所需的硬件块大小。此外，这种填充不会影响最终的注意力计算，因为我们在 *Softmax* 计算期间显式地将填充区域中的值设置为零。这确保了填充值被有效地从注意力分数中排除。此外，由于在 $\text{Attention score} \times \text{value}$ 操作期间填充的数据被减少，因此它不会影响最终输出的形状。

因此，可以有效地支持任意形状的矩阵乘法，而无需为 N 和 K 维度引入额外的填充开销。

5.3 处理动态形状

鉴于我们专注于固定形状，例如平铺大小的倍数，并且在预算限制下运行，我们缩小了处理的代币大小的范围。虽然令牌计数在不同的工作负载中可能会有很大差异，但我们应用了一种平滑技术，可以将涉及的形状减少到固定的、明确定义的大小。这使我们能够对这些特定形状进行离线定制和优化，特别是难以静态实现的优化，例如 *swizzling* [15]。

Swizzling 是一种通过改变 AI 核心之间的任务分配顺序来优化矩阵乘法性能的技术，从而提高 L2 缓存命中率。然而，确定 *swizzling* 的最优分配策略是复杂的，并且不能通过理论公式轻松推导。

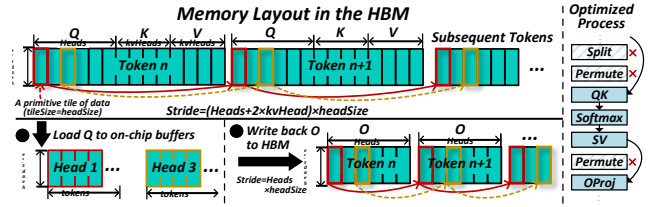


图 13：消除内存操作。

为了应对这一挑战，我们进行离线分析以探索可能的访问模式并确定最有效的核心间分发策略。

这些最佳配置被存储以供将来使用。在在线执行期间，我们将动态工作负载转换为最接近的支持的固定形状。使用这个固定形状，我们从离线分析结果中检索预先优化的配置。这种方法可确保以最佳配置处理动态工作负载，从而在执行过程中实现最佳性能。通过利用静态优化，即使面对动态工作负载，我们也可以充分发挥硬件的潜力。

5.4 消除内存开销

对于 QKV 输入，其形状为 $[\text{tokens}, \text{heads} + 2 \times \text{kvHeads}, \text{headSize}]$ ，其中最外层维度对应于标记，每个标记包含其 QKV 融合。注意操作是沿着 *Head* 维度并行执行的，这需要将 *Head* 维度定位为最外层维度。此外，我们需要分别读取 Q 、 K 和 V ，而不是融合的 QKV 张量。通常，引入 *Split* 和 *Permute* 操作来重新定向张量以实现高效计算。在注意力计算之后，应用另一个 *Permute* 操作将输出 O 转换回形状 $[\text{tokens}, \text{heads}, \text{headSize}]$ 。

为了减少内存开销，我们将 *Split* 和 *Permute* 直接融合到如图 13 所示的 GEMM 计算中。对于 QK 操作，采用基于图块和基于步幅的读取直接从 QKV 张量访问所需的数据，从而无需单独的 *Split* 和 *Permute* 步骤。类似地，对于 SV 计算，使用基于图块和基于步幅的写入以与 *OPProj* 直接对齐的格式存储结果。此集成删除了显式的 *Permute* 运算，允许 SV 输出无缝流入后续矩阵乘法运算。

6 评价

6.1 XY-服务实施

我们基于 vLLM [31] 构建了一个 Ascend 原生推理系统，利用 Ascend 内在实现核心模块

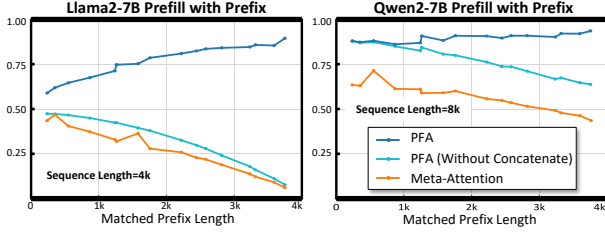


图 14: 带前缀的预填充注意力的性能。

SmoothGEMM、Meta-Attention 等规则，以及标准化、激活和嵌入等其他基本运算符。这些算子通过pybind11[7]暴露给Python API，并无缝替换vLLM中相应的GPU内核，使vLLM能够支持Ascend NPU。

为了减少频繁 Python 调用的开销，我们将整个模型转发过程卸载到 C++，将优化的运算符集成到单个 C++ 函数中。然后将该模型转发函数暴露给 vLLM 进行调用，确保简化且高效的执行路径。

此外，我们用 token-wise 调度策略替换了原生 vLLM 调度程序，集成了工作负载分解和计算任务重新排序，以更好地支持动态工作负载。我们重新设计了 vLLM 中的推测解码框架，支持元注意力的令牌树构建和元数据生成。这些增强功能使我们能够实现基于树的推测解码算法，例如 Lookahead Decoding [46]，进一步提高推理性能。

6.2 元注意力的表现

在本节中，我们评估了现实系统中通常遇到的动态工作负载下注意力内核的性能。比较目标是来自 torch-npu 2.1 [6] 的 PromptFlashAttention (PFA) [12] 和 InceFlashAttention (IFA) [11]。

6.2.1 使用任意前缀预填充Attention

在实际系统中，匹配的系统前缀的长度可以任意变化。因此，评估任意长度前缀重用下的性能至关重要。为了模拟这种行为，我们逐个调整输入提示的重用令牌数量，并评估匹配的不同长度的系统前缀下的性能。图 14 显示了 4k 和 8k 提示输入在不同系统前缀重用（范围从 0 到 4k）下的性能。结果表明，随着系统前缀的增加，我们的元注意内核的处理时间减少。然而，PFA 内核并没有从前缀重用中受益，主要是因为它的预填充内核不支持 PagedAttention 并且只接受连续的 Q 、 K 和 V 。当我们连接前缀命中时

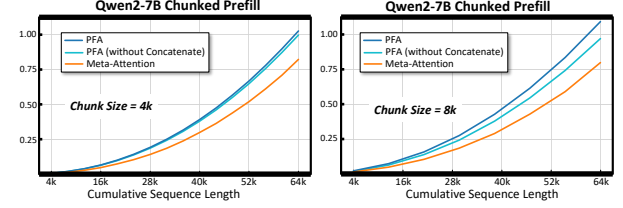


图 15: 带有分块预填充的长序列注意力。

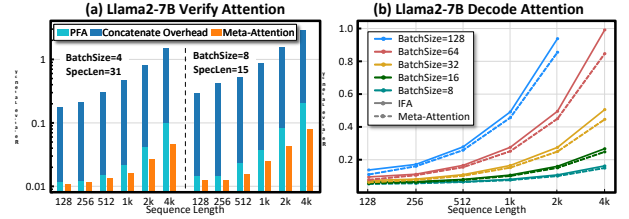


图 16: LLM 验证和解码注意力的性能。

从具有新的 K 和 V 的 K/V 缓存中，串联时间随着重用长度的增长而增加，抵消了前缀重用的好处。即使比较 PFA 内核的计算时间（不包括 K/V 连接过程），我们的内核的平均性能提高了 22.4%。

6.2.2 长序列分块预填充

为了处理长序列，将序列分块成更小的片段是一种广泛使用的方法。一方面，分块通过将其与管道并行性相结合来实现序列并行化[16, 39]。另一方面，它减少了预填充对解码中断的影响[17]。分块预填充方法将长序列分割成多个块，然后按顺序处理它们。处理完每个块后，相应的键和值将存储在 K/V 缓存中，以供后续块重用。图 15 显示了我们的分块预填充方法对于长序列的性能（块大小设置为 4k 和 8k，序列长度最大为 64k）。结果表明，我们的性能在所有序列长度上都超过了 PFA。即使将纯计算时间与 PFA 进行比较，我们的内核也显示出高达 22.2% 的改进。

6.2.3 推测解码

接下来，我们评估不同上下文长度下验证内核的性能。我们将性能与 $batchSize = 4$ 、 $specLen = 31$ 和 $batchSize = 8$ 、 $specLen = 15$ 进行比较。图 16(a) 显示，在不同的上下文长度上，我们的内核始终优于 PFA。此外，随着上下文长度的增加，随着历史长度的增加，性能改进变得越来越耗时。即使排除此串联操作

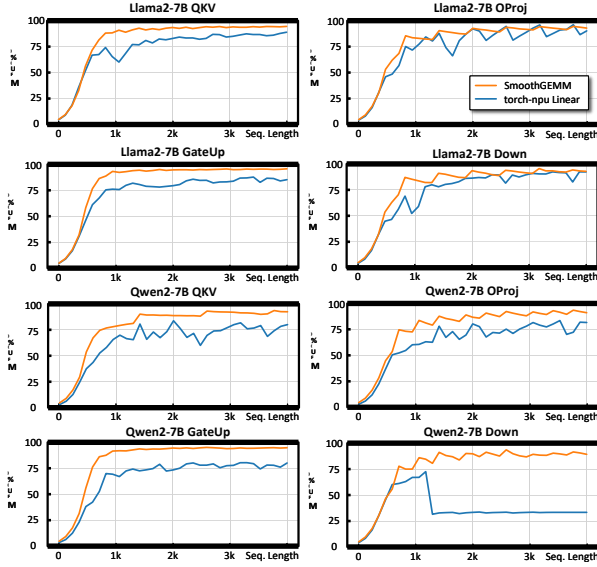


图 17: 线性 GEMM 性能。

根据 PFA 并仅比较纯计算时间，我们的内核仍然表现出 28.6% 的平均改进。

6.2.4 解码性能

在解码阶段，上下文长度和批量大小都可以任意变化。为了灵活支持任意上下文长度的解码，我们启用了 Page dAttention 优化。为了评估这种条件下的解码性能，我们测量了不同上下文长度和批量大小的性能。图 16 显示了 Llama2-7B 在不同序列长度和批量大小下的性能。结果表明，与 IFA 内核相比，我们的内核在批量大小和序列长度的所有组合上都实现了性能改进，平均提高了 12.9%。

6.3 SmoothGEMM的性能

在实际系统中，用户输入的长度可以任意变化，范围从 0 到模型支持的最大长度。对于长序列，为了最小化预填充对解码的影响并优化序列并行化，我们通常采用分块预填充策略，该策略对最大块长度施加限制，例如 4k。在现实场景中，也可能会遇到小于块大小的长度。为了评估不同条件下的性能，我们使用输入长度范围从 1 到块大小 (4096) 的 LLM 进行评估。

我们使用从 Llama2-7B 和 Qwen2-7B 导出的形状与 TP=1 比较线性算子 (QKV、OProj、GateUp 和 Down) 的性能。如图 17 所示，结果表明 SmoothGEMM 的性能比 torch-npu 线性平均高出 14.6%，表现出优越性

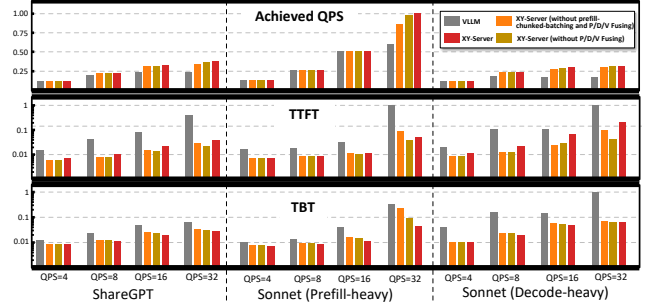


图 18: 夜间基准的端到端评估。

几乎所有测试形状的性能。此外，性能保持稳定，通常在序列大小高于 1k 时实现理想的 MFU。

6.4 端到端评估

6.4.1 vLLM 每晚基准

对于端到端基准测试，我们使用 vLLM 社区 [13] 的夜间基准测试，以 Qwen2-7B 作为模型。基线比较是针对支持 Ascend NPU 的 vLLM 社区版本 (Ascend-vLLM) [14]，主要利用 torch-npu 和 Fused-Attention 算子提供的 GEMM 进行计算。测试数据分为三种场景：ShareGPT、Prefill-heavy (462 个输入令牌和 16 个输出令牌) 和 Decode-heavy (462 个输入令牌和 256 个输出令牌)。

如图 18 所示，我们对每个测试数据集在每秒固定请求率 (QPS) 4、8、16 和 32 的情况下测量性能。收集以下指标：平均首次令牌时间 (TTFT)、平均令牌间隔时间 (TBT) 和实现的 QPS。即使没有启用预填充分块批处理和 P/D/V 融合等高级功能，XY-Serve 也比基准表现出明显的性能优势。具体而言，XY-Serve 在各种工作负载类型中实现了高达 79% 的 QPS 改进。此外，它的平均 TTFT 延迟降低了 64%，平均 TBT 延迟降低了 57%。这一改进主要归功于算子实现的有效优化。

通过启用预填充分块批处理和 PDV 融合的动态调度优化，XY-Serve 在所有场景中进一步将 QPS 提高了 89%，并将平均 TBT 延迟降低了 69%。这一结果强调了 XY-Serve 对动态工作负载的强大支持，有效地受益于这些增强功能。

当启用预填充分块批处理时，每次预填充中处理的令牌长度有效地保持在预算长度，从而提高了 MFU 并减少了高压条件下的 TTFT。然而，启用 P/D/V 融合会导致在高吞吐量压力下的解码繁重场景中 TTFT 延迟略有恶化。在这种情况下，

与预填充融合的解码数量增加，这对 TTFT 略有影响。

6.4.2 升腾NPU对比。GPU

我们比较了 910B 上运行的 XY-Serve 与 Nvidia A800 上运行的官方 vLLM-v0.6.4.post1 之间的端到端推理 MFU 和 MBU。这些测量是在 Qwen2-7B 和 Llama2-7B 模型在 TP=1 的整个前向传递的预填充和解码阶段进行的，跨不同的序列长度。如图19所示，XY-Serve实现了与A800类似的MFU和MBU。值得注意的是，在MBU方面，XY-Serve比GPU表现出明显的优势，提升高达17%。

7 相关工作

注意力优化：FlashAttention1 [24] 和 FlashAttention2 [23] 通过平铺计算来优化预填充阶段，以避免 HBM 访问，从而提高性能。FlashAttention3[40]通过softmax和矩阵运算之间的并行性进一步增强了性能。FastAttention [36] 将 FlashAttention2 从 GPU 扩展到 Ascend NPU，而 FlashDecoding [5] 通过沿序列维度分割来提高小批量的解码效率。最近的工作[2,43]通过在共享前缀时将GEMV操作转换为GEMM操作来进一步优化解码性能。虽然这些技术主要针对预填充或解码阶段，但 POD-Attention [29] 同时优化这两个阶段，从而最大化计算能力和带宽。相比之下，我们的工作通过动态前缀重用和推测算法来处理现实世界的部署场景，从而导致混合的 P/D/V 阶段。我们将动态工作负载分解为硬件友好的元基元，简化了注意力模块设计。

线性优化：现有技术如 swizzling [15]、split-k [1] 和 ping-pong [3] 常用于线性优化。我们的方法表明，支持特定的矩阵形状足以满足动态 LLM 工作负载。通过使用上述技术离线优化这些形状，我们存储配置并在线执行期间应用它们以实现最佳性能。

服务系统：一些工作旨在解决推理系统的动态性质。Orca [44] 通过迭代级调度减轻输出长度的可变性，而 PageAttention [31] 优化 KV 缓存的内存分配，减少固定长度分配浪费。我们的方法建立在这些基础上，解决现实世界推理系统中的额外复杂性，例如混合 P/D/V 阶段和每个阶段的动态形状。

解码时预填充的中断会增加 TBT。已经提出了两种策略来解决这个问题：SplitFuse [17,18,26]，它将预填充阶段划分为更小的块，并使用解码和分解的 LLM 来融合块 [27]，

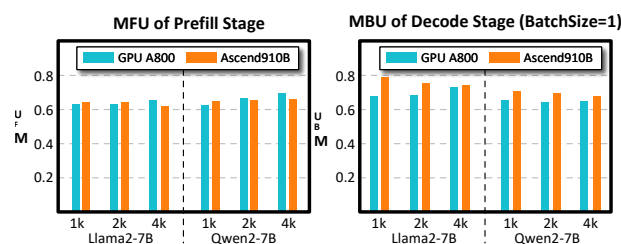


图 19：升腾 NPU 和 GPU 之间的比较。

28, 39, 48]，它在不同的机器上分开预填充和解码。XY-Serve 支持两种部署。它还可以在分类设置中实现预填充和解码角色之间的无缝转换。

8 结论

我们推出了 XY-Serve，这是一种端到端生产服务系统，旨在应对动态 LLM 工作负载的挑战。通过集成工作负载分解、计算任务重新排序和元内核（Meta-Attention 和 SmoothGEMM），XY-Serve 优化了现实生产环境中的吞吐量、延迟和计算效率。实验结果表明，Ascend NPU 具有卓越的性能，可提供与 GPU 相当的 MFU 和 MBU。凭借其处理各种动态工作负载的灵活性，XY-Serve 为生产级 LLM 推理系统的效率和适应性树立了新的基准。

参考

- [1] 使用 Triton 内核加速 Llama3 FP8 推理。 https://pytorch.org/blog/accelerating-llama3/?hss_channel=lcp-78618366/.
- [2] 使用 FlashInfer | FlashInfer 加速 LLM 服务的自我注意力。 <https://flashinfer.ai/2024/02/02/introduce-flashinfer.html>.
- [3] 深入探讨 CUTLASS Ping-Pong GEMM 内核 | PyTorch。 <https://pytorch.org/blog/cutlass-ping-pong-gemm-kernel/>.
- [4] 通过自推测解码更快地生成文本。 <https://huggingface.co/blog/layerskip>.
- [5] 用于长上下文推理的 Flash-Decoding。 <https://pytorch.org/blog/flash-decoding/>.
- [6] GitHub - Ascend/pytorch: Ascend PyTorch 适配器 (torch_npu)。 <https://gitee.com/ascend/pytorch> 的镜像。 <https://github.com/Ascend/pytorch>.

- [7] GitHub - pybind/pybind11: C++11 和 Python 之间的无缝可操作性。 <https://github.com/pybind/pybind11>。
- [8] 简介——vLLM。 https://docs.vllm.ai/en/latest/automatic_prefix_caching/apc.html。
- [9] Torch.nn-原生PyTorch API-PyTorch2.1-API列表-PyTorch网络模型移植与训练指南-模型开发 (PyTorch) -7.0.0-CANN商业版-Ascend文档-Ascend社区。 https://www.hiascend.com/document/detail/en/canncommercial/700/modeldevpt/ptmigr/ptaoplist_000006.html。
- [10] Torch_npu.npu_fused_infer_attention_score。 https://www.hiascend.com/doc_center/source/zh/Pytorch/60RC2/apiref/apilist/ptaoplist_000787.html。
- [11] Torch_npu.npu_incre_flash_attention。 https://www.hiascend.com/doc_center/source/zh/Pytorch/60RC2/apiref/apilist/ptaoplist_000788.html。
- [12] Torch_npu.npu_prompt_flash_attention。 https://www.hiascend.com/doc_center/source/zh/CANNCommunityEdition/80RC1alpha001/apiref/fmkadptapi/ptaoplist_000142.html。
- [13] vllm 每晚基准。 <https://github.com/vllm-project/vllm/tree/main/buildkite/nightly-benchmarks>。
- [14] vllm 对 ascend npu 的支持。 <https://github.com/vllm-project/vllm/pull/8054>。
- [15] 使用线程组 ID Swizzling 优化 L2 局部性的计算着色器。 <https://developer.nvidia.com/blog/optimizing-compute-shaders-for-l2-locality-using-thread-group-id-swizzling/>, 2020 年 7 月。
- [16] Amey Agrawal, Junda Chen, Íñigo Goiri, Ramachandran Ramjee, Chaojie 张, Alexey Tumanov 和 Esha Choukse。 Mnemosyne: 并行化策略, 无需近似即可有效地服务数百万上下文长度 llm 推理请求。 *arXiv preprint arXiv:2409.17264*, 2024 年。
- [17] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov 和 Ramachandran Ramjee。 使用 sarathi-serve 控制 llm 推理中的吞吐量和延迟权衡。 *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 第 117-134 页, 2024 年。
- [18] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani 和 Ramachandran Ramjee。 Sarathi: 通过使用分块预填充进行解码, 实现高效的 llm 推理。 *arXiv preprint arXiv:2308.16369*, 2023 年。
- [19] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever 和 Dario Amodei。 语言模型是小样本学习者。 *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS '20*, 美国纽约州雷德胡克, 2020 年。 Curran Associates Inc.
- [20] 蔡天乐, 李玉红, 耿正阳, 彭洪武, Jason D Lee, 陈德明, Tri Dao。 Medusa: 具有多个解码头的简单 llm 推理加速框架, 2024 年。 [URL https://arxiv.org/abs/2401.10774](https://arxiv.org/abs/2401.10774)。
- [21] 尼古拉斯·卡里昂, 弗朗西斯科·马萨, 加布里埃尔·辛奈夫, 尼古拉斯·乌苏尼尔, 亚历山大·基里洛夫和谢尔盖·扎戈鲁伊科。 使用转换器进行端到端对象检测。在 *European conference on computer vision* 中, 第 213-229 页。施普林格, 2020。
- [22] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre 和 John Jumper。 通过推测采样加速大型语言模型解码。 *arXiv preprint arXiv:2302.01318*, 2023 年。
- [23] 三道。 Flashattention-2: 更快的注意力, 更好的并行性和工作分区 (2023)。 *arXiv preprint arXiv:2307.08691*, 2023 年。
- [24] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra 和 Christopher Ré。 Flashattention: 具有 io 意识的快速、内存高效的精确注意力。 *Advances in Neural Information Processing Systems*, 35: 16344–16359, 2022。
- [25] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit 和 Neil Houlsby。 一张图像相当于 16x16 个单词: 用于大规模图像识别的 Transformer。在 *International Conference on Learning Representations*, 2021 年。

- [26] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyangqin, Arash Bakhtiari, Lev Kurilenko 等。Deepspeed-fastgen: 通过 mii 和 deepspeed-inference 为 llms 生成高吞吐量文本。 *arXiv preprint arXiv:2401.08671*, 2024 年。
- [27] 胡存辰, 黄河阳, 徐亮亮, 陈旭升, 徐江, 陈爽, 冯浩, 王晨曦, 王飒, 鲍云刚, 等。无干扰推理: 针对混合下游工作负载分解 llm 推理。 *arXiv preprint arXiv:2401.11181*, 2024 年。
- [28] 金一波, 王涛, 林惠民, 宋明阳, 李培阳, 马一鹏, 单一成, 袁正凡, 李彩龙, 孙雅静, 等。P/d-serve: 大规模服务分类的大型语言模型。 *arXiv preprint arXiv:2408.08147*, 2024 年。
- [29] Aditya K Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee 和 Ashish Panwar。Pod-attention: 解锁完整的预填充解码重叠, 以实现更快的 llm 推理。 *arXiv preprint arXiv:2410.18038*, 2024 年。
- [30] 贾里德·卡普兰、萨姆·麦坎德利什、汤姆·赫尼汉、汤姆·B·布朗、本杰明·切斯、雷旺·柴尔德、斯科特·格雷、亚历克·雷德福德、杰弗里·吴和达里奥·阿莫迪。神经语言模型的缩放定律。 *arXiv preprint arXiv:2001.08361*, 2020。
- [31] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Yingheng, Lianmin Cheng, Codyhao Yu, Joseph Gonzalez, Hao Zhang, Ion Stoica。针对页面数据注意力服务的大型语言模型的高效内存管理。 *Proceedings of the 29th Symposium on Operating Systems Principles*, 第 611-626 页, 2023 年。
- [32] Yaniv Leviathan, Matan Kalman 和 Yossi Matias。通过推测解码从 Transformer 进行快速推理。在 *International Conference on Machine Learning* 中, 第 19274-19286 页。PMLR, 2023。
- [33] 李玉辉, 魏方云, 张超, 张红阳。Eagle: 推测性抽样需要重新思考特征的不确定性。 *arXiv preprint arXiv:2401.15077*, 2024 年。
- [34] 廖恒, 屠家进, 夏静, 刘虎, 周西平, 袁洪辉, 胡玉兴。Ascend: 用于无处不在的深度神经网络计算的可扩展且统一的架构: 行业跟踪论文。 *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 第 789-801 页。IEEE, 2021。
- [35] 廖恒, 屠家进, 夏静, 周西平。达芬奇: 神经网络计算的可扩展架构。在 *2019 IEEE Hot Chips 31 Symposium (HCS)* 中, 第 1-44 页。IEEE 计算机协会, 2019 年。
- [36] 林浩然, 余宪志, 赵康, 侯鲁, 詹宗元, 斯坦尼斯拉夫·加米涅夫, 韩包, 胡婷, 王明凯, 常启新, 等。Fastattention: 将 flashattention2 扩展到 npu 和低资源 GPU。 *arXiv preprint arXiv:2410.16663*, 2024 年。
- [37] 刘晓轩, Cade Daniel, 胡郎翔, Woosuk Kwon, 李卓瀚, 莫湘西, Alvin Cheung, 邓志杰, Ion Stoica, 张浩。使用良好吞吐量优化推测解码以服务于大型语言模型。 *arXiv preprint arXiv:2406.14066*, 2024 年。
- [38] 苗旭鹏, Gabriele Oliaro, 张志豪, 程新浩, 王泽宇, 张正新, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, 石小翔, 等。Specinfer: 通过基于树的推测推理和验证来加速大型语言模型。 *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 第 932-949 页, 2024 年。
- [39] 秦若愚, 李哲明, 何蔚然, 张明星, 吴永伟, 郑伟民, 徐欣然。Mooncake: 用于 llm 服务的以 kvcache 为中心的分解架构。 *arXiv preprint arXiv:2407.00079*, 2024 年。
- [40] Jay Shah, Ganesh Bikshandi, 张英, Vijay Thakkar, Pradeep Ramani 和 Tri Dao。Flashattention-3: 快速准确的注意力, 异步且精度低。 *arXiv preprint arXiv:2407.08608*, 2024 年。
- [41] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale 等。Llama 2: 开放基础和微调的聊天模型。 *arXiv preprint arXiv:2307.09288*, 2023 年。
- [42] 安阳, 杨宝松, 惠斌源, 郑波, 于博文, 常周, 李成鹏, 李成元, 刘大一恒, 黄飞, 等。Qwen2 技术报告。 *arXiv preprint arXiv:2407.10671*, 2024 年。
- [43] 陆野, 泽涛, 黄勇, 杨力。ChunkAttention: 具有前缀感知 KV 缓存和两阶段分区的高效自注意力。载于 Lun-Wei Ku, Andre Martins 和 Vivek Srikumar, 编辑, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 第 11608-11620 页, 泰国曼谷, 2024 年 8 月。计算语言学协会。

[44] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soo-jeong Kim 和 Byung-Gon Chun。Orca: 基于变压器的生成模型的分布式服务系统。

16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), 第 521-538 页, 2022 年。

[45] 张军, 王珏, 李欢, 寿力丹, 陈科, 陈刚, Sharad Mehrotra。起草和验证: 通过自推测解码进行无损大语言模型加速。 *arXiv preprint arXiv:2309.08168*, 2023 年。

[46] 赵耀, 谢志田, 陈亮, 庄陈毅, 顾金杰。Lookahead: 具有无损生成精度的大型语言模型的推理加速框架。
。 *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 第 6344-6355 页, 2024 年。

[47] 郑连民, 尹良胜, 谢志强, 孙初跃, 黄杰夫, 于浩, 曹诗一, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, 等。Sglang: 结构化语言模型程序的高效执行, 2024 年。URL <https://arxiv.org/abs/2312.07104>。

[48] 钟银民, 刘胜宇, 陈俊达, 胡建波, 朱一波, 刘煊哲, 金鑫, 张浩。Dist-serve: 分解预填充和解码, 以实现良好输出优化的大型语言模型服务。 *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 第 193-210 页, 2024 年。